

Technical Concept: Hábito Habit Tracker

Phase 1 System Architecture & Specifications

1. Intro

As part of the course DLBDSOOFPP01, the task to create a habit tracker is given. In phase 1, the object is to create a technical concept of the program and hand it in. This document serves as the formal technical concept for Phase 1.

2. Pitch

Imagine working an interesting, challenging job. Every day you give your very best, yet still in the evenings you feel somewhat tired and exhausted. You think, it is just the stress, it will go away.

After some weeks, you figure you are not going to sports anymore, and rarely you take the time to meditate.

"You will do it tomorrow."

Another tomorrow, and still, nothing. Feels like you just did it yesterday, but also feels like you never gonna do it again.

Wouldn't it be great, if you could keep track of your habits? Visualizing behaviour can be a first step into recognizing patterns and think of actionable items.

This is where Hábito, your personal habit tracker, comes into play. Supporting you to log your habits, whether they are daily or weekly. But it is not only logging of course—to give you that extra push of motivation, you can also track your streaks. And for the inner monk, you can access all your data via the analytics module to see patterns clearly.

3. Technical Concept

3.1 Implementation Constraints

The course is called 'Object Oriented and Functional Programming with Python' and as such the implementation needs to be based on modern Python. The core runtime environment version is explicitly specified to be ≥ 3.7 .

3.2 Acceptance Criteria

- Comprehensive documentation (including installation instructions, run instructions, and test execution instructions).
- Pervasive usage of Python docstrings for code-level documentation.
- Usage of clean Python class(es) for the core implementation.
- Supporting at least weekly and daily habit tracking periodicities.
- Shipped with out-of-the-box demo data (containing exactly 5 predefined habits and example tracking data spanning the past 4 weeks).
- Automated timestamps for both habit creation and completion actions.
- Robust data persistence mechanism.
- Analytics module implemented using the functional programming paradigm:
 - List of all currently tracked habits
 - List of all habits with the same periodicity
 - Longest run streak of all defined habits
 - Longest run streak for a specific habit
- An intuitive, self-explanatory user interface.
- A comprehensive unit test suite to ensure functional regression safety.

4. Design Considerations

4.1 User Perspective

As a user, I don't want complex UI frameworks to get in my way. Since *In the Beginning there was Command Line* [1], the true functional beauty of an engineering tool can be expressed most cleanly directly on the CLI.

4.2 Technical Decisions

4.2.1 User Interface

A pure input loop might feel a bit too restrictive or spartan for the modern command line warrior. To bridge this gap, we will use the Rich package to support a stylized, visually rich CLI environment inside the terminal container.

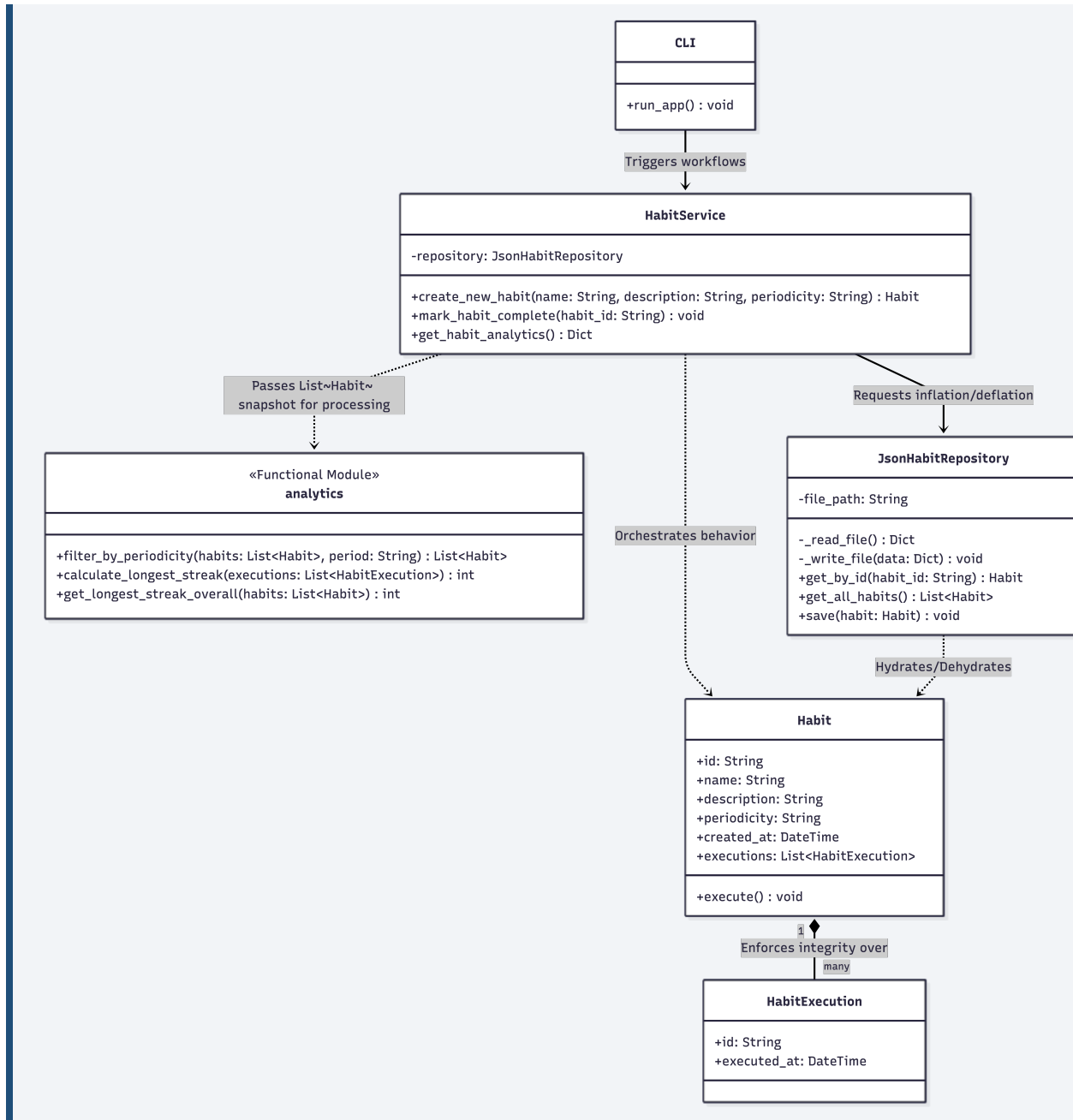
4.2.2 Multi-Device Support & Interchangeability

"Everything is a file" is a proven foundational design pattern in UNIX-like systems. Following this elegant philosophy, Hábito uses plain, human-readable JSON files as its main storage tier. This enables users to source-control data via Git, synchronize smoothly across devices, and protects users from any vendor or tool lock-in.

5. Technical Design

5.1 System Architecture & Class Diagram

The diagram below outlines the structural boundaries of Hábito. While core data mutations and lifecycle operations are strictly managed using Object-Oriented principles, the analytics module is isolated as a stateless, pure functional layer to fulfill functional programming criteria without side effects.



5.2 Architectural Justification

- **High Testability:** Isolating pure domain operations ensures that critical tracking components can be rigorously verified via unit tests without mocking file-system dependencies.
- **Infrastructure Decoupling (Swapability):** By utilizing the Repository Pattern, the core domain remains entirely oblivious to data persistence mechanisms. This allows the data backend to be seamlessly upgraded from a flat JSON file to a relational database (e.g., SQLite3) in the future without altering business logic.
- **Presentation Extensibility:** Separating the CLI layer ensures that alternative entry points, such as a Flask-based or FastAPI REST API, can be cleanly attached later without modifying underlying application workflows.

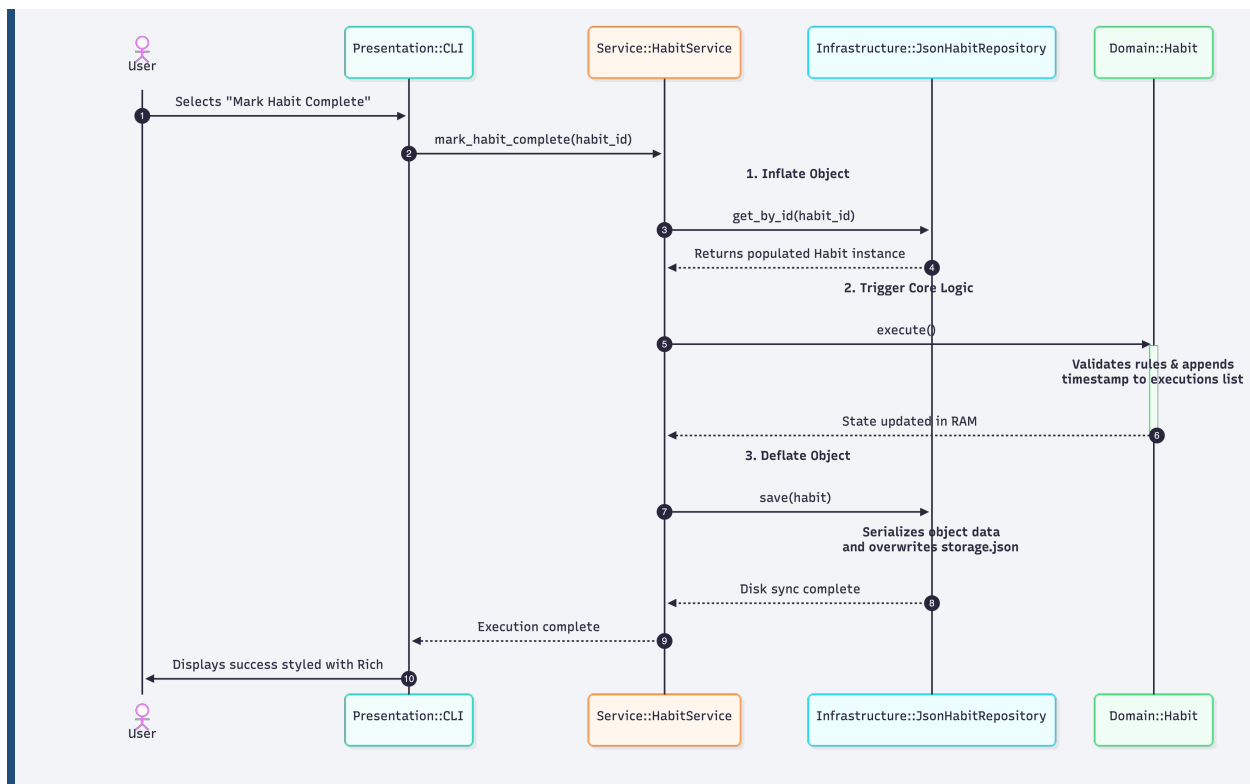
5.3 Tool Justification

- **Python json Module (Persistence):** Provides an interchangeable, human-readable, and machine-readable snapshot format that prevents platform lock-in and allows seamless manual inspection of data fixtures.
- **Rich Library (Presentation Layer):** Abstracts terminal rendering complexities, enabling the delivery of a clean, structured interface featuring stylized menus and live data tables without polluting domain code with low-level console formatting escape sequences.
- **Python datetime Module (Domain Logic):** Essential for converting string-based timestamps into rich temporal objects in RAM. This allows the domain layer to reliably compute complex calendar mathematics, such as consecutive-period tracking and streak boundaries, safely.

5.4 Application Process & User Flow

The execution flow of Hábito follows a strict linear path downward through the architectural layers to guarantee data isolation and predictability:

- **Interaction (Presentation Layer):** The user selects an option from the Rich command line interface menu, such as marking a habit as complete.
- **Orchestration (Service Layer):** The CLI invokes the HabitService controller. The service acts as the orchestrator; it coordinates with the JsonHabitRepository to locate the habit data on the disk and "inflate" it into a living Python object in memory.
- **Validation & State Change (Domain Layer):** The service delegates the execution to the Habit entity by calling `.execute()`. The entity evaluates internal business logic rules (e.g., preventing duplicate check-offs within the same period) and alters its internal executions collection.
- **Persistence (Infrastructure Layer):** Once the domain entity successfully mutates its state in RAM, the service passes the updated object back to the JsonHabitRepository. The repository serializes ("deflates") the entity state back into raw JSON data and flushes it to the storage file on disk.
- **Feedback Loop:** The service completes its routine and signals success back to the CLI, which prints a stylized confirmation message to the user.



[1] Stephenson, N. (1999). *In the beginning ... was the command line*. Avon Books.